TP3 – LCD Display – System Setup Menu

Objetivos

Usar **Itemis Create** **Desktop**
Generar proyecto **Itemis Create**
Editar/simular/depurar **diagramas de estado** (**modelos**)

Usar **STM32CubeIDE**
Generar proyecto **STM32**
Editar/compilar/depurar/portar programas en **C** – Codificar statecharts en **C**
Configurar el **IDE** para depurar en modo **semihosting**
Adoptar metodología orientada a la portabilidad/reúso de **código**
Identificar/implementar **Patrones de Diseño de Software**

Usar **Git** y **GitHub**
Generar/actualizar **repositorio**
Generar/actualizar **archivos** del tipo **README.md** (**Markdown**)

Usar **Gemini**
Analizar/explicar código **fuente**

Diagramas de Estado – Modelado – Codificación en C

LCD Display (porting C code)
Display Statechart (modeling & C coding)
System Setup Menu Statechart (modeling & C coding)
Model integration

Entradas/Salidas Digitales

Configuration and use (polling, interrupt)
LCD Display (4-Bit Mode) & Keypad

Materiales requeridos

1 (UN) Placa NUCLEO-F103RB
1 (UN) Cable Usb A Mini Usb - Arduino Nano
1 (UN) Teclado membrana 1 x 4 teclas - Arduino o 1 (UN) Dip switch 4 contactos
1 (UN) Módulo LED RGB - Arduino o 1 (UN) Módulo LED Mini Semáforo - Arduino
1 (UN) Display LCD 16x2 (HD44780 compatible) - Arduino

Actividades

1er encuentro

TP3-01 - LCD Display Driver (porting C code)
TP3-02 - LCD Display Statechart (modeling, C coding)

2do encuentro

TP3-03 - System Setup Menu Statechart (modeling)
TP3-04 - System Setup Menu Statechart (C coding)

Resolución en grupos de **3** alumnos, requiere **presentación** por parte de un **responsable** del grupo

---

TP3 – Actividad 01 – LCD Display Driver (porting C code)

Objetivos

Usar STM32CubeIDE

Generar proyecto STM32

Editar/compilar/depurar/portar programas en C

Analizar cumplimiento de restricciones temporales del **ejecutor cíclico**

Identificar Patrones de Diseño de Software

Diagramas de Estado – Codificación en C

Usar Git y GitHub

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (Markdown)

Usar Gemini

Analizar/explicar código **fuente**

Paso 01: **Crear** un nuevo **repositorio** en **GitHub** para almacenar su **TP3**, pasos:

GitHub: **Repositories** => **New** =>

Repository name: **tdse-tp3_Año-Cuatrimestre_Curso-Grupo**

Description: **# FIUBA - Electrónica - Taller de Sistemas Embebidos - Trabajo Práctico N°: 3 - LCD Display - System Setup Menu**

Initialize this repository with:

Tildar **Add a README file**

=> **Create repository**

> **Año-Cuatrimestre** es: **2026-1erC** o **2026-2doC** … etc. (el correspondiente al Año y Cuatrimestre de cursada)
> **Curso-Grupo** es: **1-01** o **1-02** ... etc. (el correspondiente al Curso y Grupo de cursada)

Paso 02: **Invitar** al **repositorio**, como **colaboradores** a los docentes auxiliares de su curso, responsables de la aprobación de su **TP3**, pasos:

GitHub: **Repositories** => **Settings** => **Collaborators** => Tildar: **Add people** =>

en **Find people** => tipear: **Docente_Auxiliares-FIUBA**

Tildar: **Add Docente_Auxiliares-FIUBA**

> Una vez que los docentes auxiliares de su curso, **hayan aceptado** la **invitación** a ser **colaboradores**, podrá agregarlos como **Revisores**

Paso 03: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **editar** el archivo **README.md** y **modificar** su contenido, pasos:

GitHub: **READM.me** => **Edit this file** => **borrar** su contenido => **Copiar** y **Pegar** lo siguiente:

```
# FIUBA - Electrónica - Taller de Sistemas Embebidos
## Trabajo Práctico N°: 3 - LCD Display - System Setup Menu
### Año-Cuatrimestre - Curso-Grupo

### Responsable de la entrega:
| Padrón | Apellidos, Nombres      | Fecha    | Deadline  |
| :----- | :---------------------  | :------: | :-------: |
| XXXXXX | YYYY, ZZZ               |          | Semana 08 |
```

Actualizar el archivo **README.md** con los datos del responsable de la entrega (un alumno diferente para cada TP):

Año-Cuatrimestre, **Curso-Grupo**, **Padrón**, **Apellidos** y **Nombres**

Paso 04: **Completar** TA134 - TdSE - TP1 - Registro de Entrega.

Paso 05: **Generar** un nuevo **proyecto STM32**: **descargar** el archivo **tdse-tp3_01-porting_c_code.zip** e **importar** el **proyecto STM32** que contiene: **tdse-tp3_01-porting_c_code**.

Compilar el **proyecto STM32** (**STM32CubeIDE**) importado: **tdse-tp3_01-porting_c_code**.

Confirmar que la compilación falla: **Build Failed. 14 errors, 7 warnings. (took xxxms)**, todos en el archivo **display.c**.

Paso 06: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **tdse-tp3_01-display.md**.

Paso 07: **Consultar** a **Gemini** de **Google** respecto al **TP3**, pasos:

Loguear, en **Google** con tu cuenta de correo electrónico institucional, del dominio **@fi.uba.ar**.

Ingresar a **Gemini** => https://gemini.google.com/app, => **Nueva Conversación** => **elegir** modelo **Pro** => en **Preguntar a Gemini** =>

Copiar y **Pegar** la siguiente línea de **texto**:

> **¿Puedes ayudarme a realizar un Trabajo Práctico sobre LCD Display (porting C code) - System Setup (statechart - modeling - c coding)?**

Leer y **almacenar** la respuesta en**: …\tdse_workspace\tdse-tp3_01-porting_c_code\tdse-tp3_01-display.md**.

Paso 08: **Consultar** a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Ingresar a **Gemini** => https://gemini.google.com/app, => **Nueva Conversación** => **elegir** modelo **Pro** => en **Preguntar a Gemini** =>

Copiar y **Pegar** la siguiente línea de **texto**:

Analizar y **explicar** (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: **app.c**, **app_it.c**, **systick.c**, **task_test_attribute.h**, **task_test.c**, **task_display_attribute.h**, **task_display_interface.c**, **task_display.c**, **display.h** y **display.c**.

Indicar el comportamiento de las funciones **void task_test_statechart**(**void**) y **void task_display_statechart**(**void**).

Agregar los siguientes archivos (click en **+** Agregar archivos => **Subir archivos**):

…\tdse_workspace\tdse-tp3_01-porting_c_code\app\src\app.c
…\tdse_workspace\tdse-tp3_01-porting_c_code\app\src\app_it.c

…\tdse_workspace\tdse-tp3_01-porting_c_code\app\src\systick.c

…\tdse_workspace\tdse-tp3_01-porting_c_code\app\inc\task_test_attribute.h
…\tdse_workspace\tdse-tp3_01-porting_c_code\app\src\task_test.c

…\tdse_workspace\tdse-tp3_01-porting_c_code\app\inc\task_display_attribute.h
…\tdse_workspace\tdse-tp3_01-porting_c_code\app\src\task_display_interface.c
…\tdse_workspace\tdse-tp3_01-porting_c_code\app\src\task_display.c

…\tdse_workspace\tdse-tp3_01-porting_c_code\app\inc\display.h
…\tdse_workspace\tdse-tp3_01-porting_c_code\app\src\display.c

Leer y **almacenar** la respuesta en**: …\tdse_workspace\tdse-tp3_01-porting_c_code\tdse-tp3_01-display.md**.

Documentos a tener en cuenta:

MB1136 - Electrical Schematic - STM32 Nucleo-64 boards =>
https://www.st.com/resource/en/schematic_pack/mb1136-default-c04_schematic.pdf

UM1724 - User manual - STM32 Nucleo-64 boards =>
https://www.st.com/resource/en/user_manual/um1724-stm32-nucleo64-boards-mb1136-stmicroelectronics.pdf

---

Paso 09: **Portar** fuentes **d**el archivo **display.c** del nuevo **proyecto STM32** (**STM32CubeIDE**), consiste en migrar los **accesos al hardware** del microcontrolador de **Mbed** a **STM32**, implica:

Identificar en el código fuente, los recursos de **HW** (**D4**, **D5**, **D6**, **D7**, **D8** y **D9**) & **Clases** (**DigitalOut**) & **Métodos** (**delay**(..)).

Reemplazar la instanciación de **Objetos** (**DigitalOut displayD4**(**D4**);) por la **configuración** de **tdse-tp3_01-porting_c_code.ioc** (tal como se hizo con los **GPIOs** (**botones** y **leds**) del **TP2**).

Reemplazar la gestión de **Objetos** (**displayD4** = **value**;) por **funciones** de **STM32-F1 HAL**:

```
HAL_GPIO_WritePin((GPIO_TypeDef *)D4_GPIO_Port, (uint16_t)D4_Pin, (GPIO_PinState)value);
```

Reemplazar las **demoras bloqueantes** (**delay** (**X**);) por **métodos** de **STM32-F1 HAL** o de **systick.c**.

HAL_Delay(…); (para **X** superiores a **1mS**)
systick_delay_us(**DISPLAY_DEL_37US**); (para **X** de **1mS**, excepto en la función **void displayInit**(displayConnection_t connection)

Compilar el **proyecto STM32** (**STM32CubeIDE**) portado: **tdse-tp3_01-porting_c_code**.

El archivo **display.c**, es **display.cpp** del libro **A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers** (https://www.arm.com/resources/education/books/designing-embedded-systems), **de su cuenta de GuitHub** => **Example 6.2, Use of a 4-Bit Mode to Send Commands and Data to the Display** =>
Link: https://github.com/armBookCodeExamples/example_6-2

En los ejemplos del libro, la gestión de **HW** (**periféricos**) en el código fuente **Mbed** se realiza mediante Clases de **C++**.

Para el manejar de **GPIOs** de **salida**, se recurre a la clase **DigitalOut**

Por ejemplo: **DigitalOut displayD4**(**D4**);

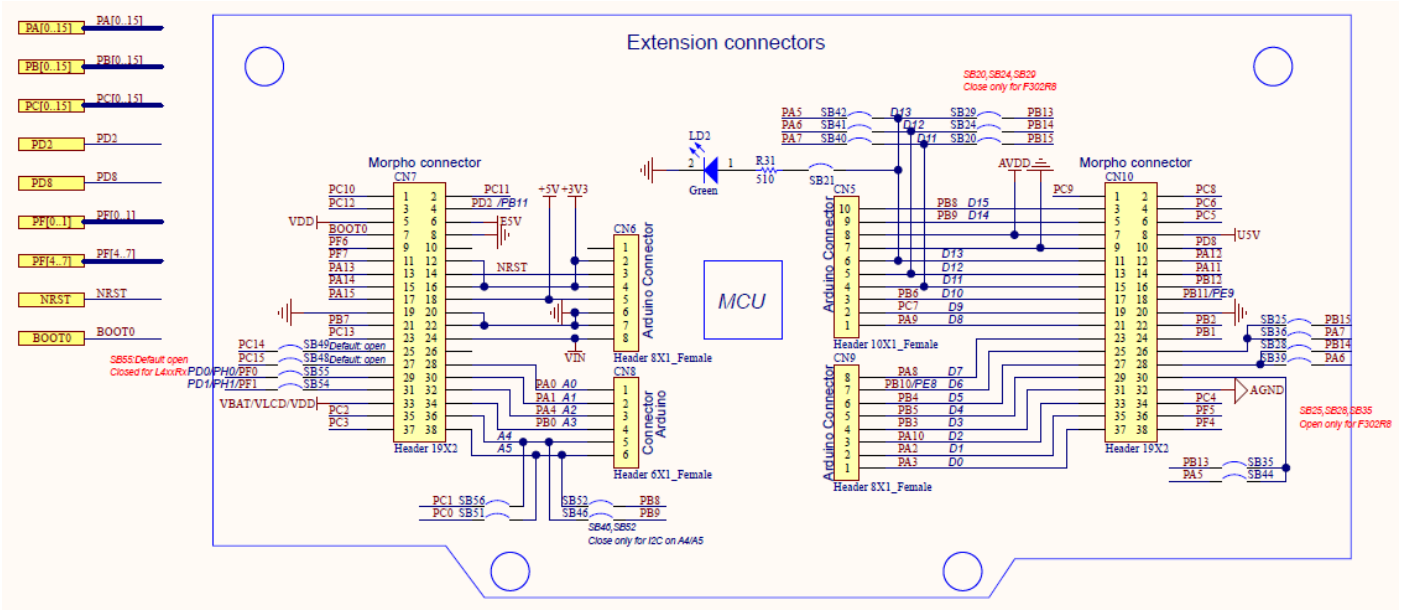
Configura como **salida** digital el pin **D4** (Arduino-compatible header: **CN9**, https://os.mbed.com/platforms/ST-Nucleo-F103RB/#:~:text=Arduino%2Dcompatible%20headers)

Por ejemplo: **displayD4** = **value**;

Escribe value en la **salida** digital **displayD4**

En el archivo **display.cpp**, se comentaron las líneas de código C relacionadas a **los pines**: **D0**, **D1**, **D2** y **D3** (innecesarios p**ara 4-Bit Mode**).

En el **TP2** vimos que los pines **D4**, **D5**, **D6**, **D7**, **D8** y **D9** están **disponibles** en la placa **NUCLEO-F103RB** (no usados).



Paso 10: **Depurar** el nuevo **proyecto STM32** (**STM32CubeIDE**):

Desconectar placa **NUCIEO-F103RB** de PC.

Conectar el **LCD Display** a la placa **NUCLEO-F103RB** como en la imagen al pie.

. No olvide conectar y ajustar el **Trimpot** (**LINEAL - 10K**) a contrast adjustment (pin: **VO**)
Tenga cuidado al conectar las líneas de **backlight** (pines: **A** & **K**

Conectar placa **NUCIEO-F103RB** a PC.

Confirmar mediante la depuración, al ejecutar el programa (**toolbar button** > [**Resume** (**F8**)]):

Que **task_test_init**(), envía a **LCD Display interface** los mensajes:

Línea 1: **"LCD Display Test"**
Línea 2: **" Porting C code "**

Que **task_test_update**(), envía a **LCD Display interface** los mensajes:

Línea 2: **"Test Nro: X*****"** (**X** se incrementa c/Segundo)

Y que dichos mensajes se ven en el **LCD Display**.

Luego de varias ejecuciones de **app_update()**, **leer** y **almacenar** los valores de **task_dta_list[index]** (**indicar** unidad de medida) en**: …\tdse_workspace\tdse-tp3_02-porting_c_code\tdse-tp3_01-display.md**. **Analizar** si cumplen con las restricciones temporales del **ejecutor cíclico**.

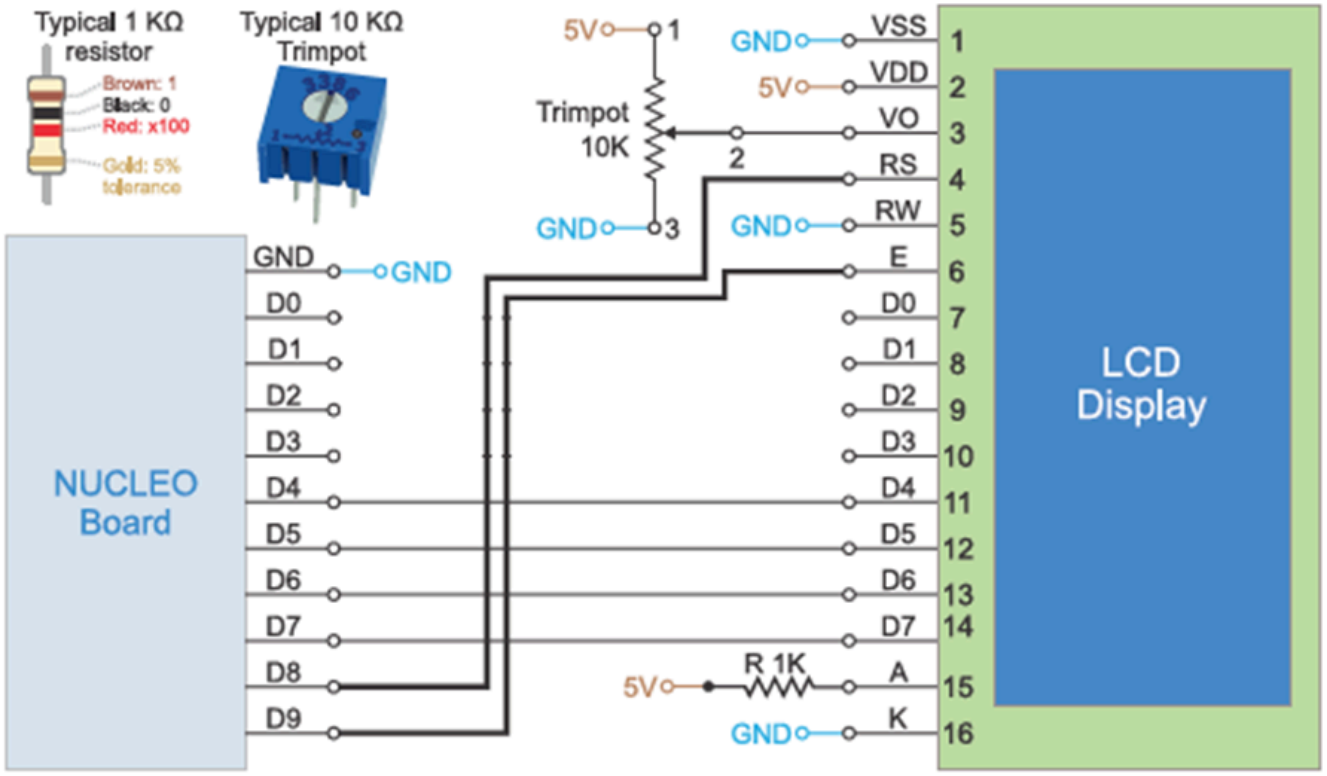


Figure 6.3 Diagram of the connections between the character LCD display and the NUCLEO board using GPIOs.

Paso 11: **Almacenar** el **proyecto STM32**: **tdse-tp3_01-portng_c_code** y el/los archivo/s de la actividad: **tdse-tp3_01-display.md**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

TP3 – Actividad 02 – LCD Display Statechart (modeling, C coding)

Objetivos

Usar STM32CubeIDE

Generar proyecto **STM32**

Editar/compilar/depurar/portar programas en **C**

Implementar Patrones de Diseño de Software

Diagramas de Estado – Codificación en C (LCD Display Statechart)

Garantizar cumplimiento de restricciones temporales del **ejecutor cíclico**

Usar Itemis Create **Desktop**

Generar proyecto **Itemis Create**
Editar/simular/depurar **diagrama de estado** (**modelo**): **Task Display Statechart** (Modeling)

Usar Gemini

Analizar/explicar código **fuente**

Paso 01: Ejecutar Itemis Create **Desktop**.

Paso 02: Generar un **modelo** nuevo, **nombre**: **task_display**.

Paso 03: **Editar** y **simular** el **modelo**, volcando allí el **comportamiento task_display** (que **escriba** en el **display 1 instrucción o 1 dato** a la vez, **c/ 1mS**).

Paso 04: **Generar** la **imagen** del **modelo** (**File** > **Save As Image File …**).

En su PC **crear** una carpeta **task_display**, **copiar** allí los archivos generados (**.ysc** y **.png**).

Paso 05: **Almacenar** la carpeta generada en la actividad **task_display**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

Paso 06: Generar un nuevo **proyecto STM32**: **copiar** el archivo **tdse-tp3_01-porting_c_code** y **pegar/renombrar** como **tdse-tp3_02-porting_c_code**.

Renombrar el archivo **tdse-tp3_01-porting_c_code.ioc** como **tdse-tp3_02-porting_c_code.ioc**.

Borrar los archivos de compilación/programación/depuración (**ttdse-tp3_01-porting_c_code.launch**, **tdse-tp3_01-porting_c_code.cfg** y **Debug** folder).

Compilar/Depurar el **proyecto STM32** (**STM32CubeIDE**) renombrado (**tdse-tp3_02-porting_c_code**).

Generar los archivos de depuración (**tdse-tp3_02-porting_c_code.launch** y **tdse-tp3_02-porting_c_code.cfg**).

Paso 07: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **tdse-tp3_02-display.md**.

Paso 09: Modificar el código fuente, del nuevo **proyecto STM32**, **codificar** en **C** una función pública **void displayDataWrite**(**const char** data), para **escribir** en el **display 1 dato**.

Para **Editar**, en **Project Explorer** view:

…\tdse_workspace\tdse-tp3_2-porting_c_code\app\src\display.c (double-click)

…\tdse_workspace\tdse-tp3_2-porting_c_code\app\inc\display.h (double-click)

Paso 09: Modificar el código fuente, del nuevo **proyecto STM32**, **codificar** en **C** el Diagrama de Estado del **modelo task_display**, (que **escriba** en el **display 1 instrucción o 1 dato** a la vez, **c/ 1mS**), tal como se describe en: **task_display.jpg**. generado en el **TP3 – Actividad 02 – Paso 04**.

Para **Editar**, en **Project Explorer** view:

…\tdse_workspace\tdse-tp3_2-porting_c_code\app\src\task_display.c (double-click)

…\tdse_workspace\tdse-tp3_2-porting_c_code\app\inc\task_display_attribute.h (double-click)

Paso 10: **Depurar** el nuevo **proyecto STM32** (**STM32CubeIDE**):

Confirmar mediante la depuración, el correcto funcionamiento.

Luego de varias ejecuciones de **app_update()**, **leer** y **almacenar** los valores de **task_dta_list[index]** (**indicar** unidad de medida), en**: …\tdse_workspace\tdse-tp3_02-porting_c_code\tdse-tp3_02-display.md**.

Analizar si cumplen con las restricciones temporales del **ejecutor cíclico**

Paso 11: **Almacenar** el **proyecto STM32**: **tdse-tp3_02-portng_c_code** y el/los archivo/s de la actividad: **tdse-tp3_02-display.md**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

TP3 – Actividad 03 – System Setup Menu Statechart (modeling)

Objetivos

Usar Itemis Create Desktop

Generar proyecto **Itemis Create**

Editar/simular/depurar diagrama de estado (**modelo**): **System Setup Menu Statechart** (Modeling)

Usar Git y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Paso 01: Ejecutar Itemis Create Desktop.

Paso 02: Generar un **modelo** nuevo, **nombre**: **system_setup_menu**.

Paso 03: **Editar** y **simular** el **modelo**, volcando allí el **comportamiento system_setup_menu**(**imagen al pie**).

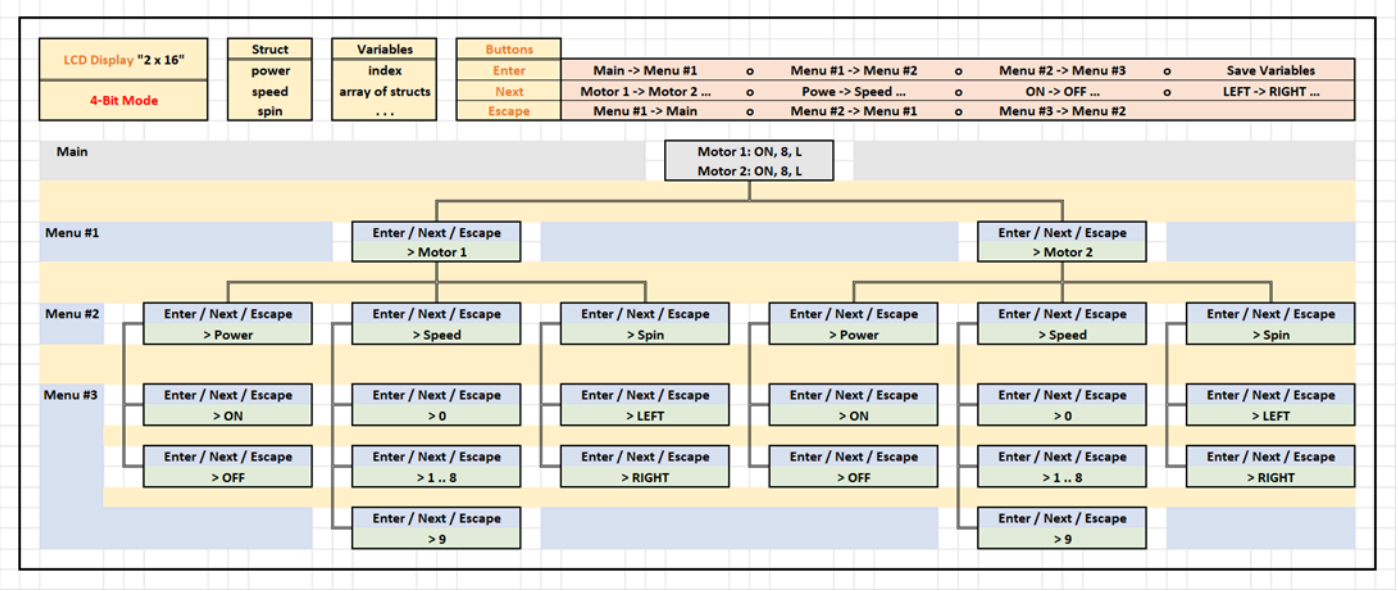
El mismo tiene **4 niveles** de menú (**Main**, **Menu #1**, **Menu #2** y **Menu #3**).

Cada nivel de menú tiene una o más **opciones** (**Main**: **1 opción**, **Menu #1**: **2 opciones**, **Menu #2**: **3 opciones**, **Menu #3**: **2 opciones** o **10 opciones**).

Para **cambiar** de **nivel** (verticalmente), cuenta con **3 botones** (**Buttons**: **Enter**, **Next** y **Escape**).

Para **cambiar** de **opción** en un **nivel** determinado (horizontalmente), cuenta con **3 botones** (**Buttons**: **Enter**, **Next** y **Escape**).

Los **3 botones** (**Buttons**: **Enter**, **Next** y **Escape**) generan los **eventos** del **modelo System Setup**.



Puede recurrir a **variables**, **estructuras** y **arreglos de estructuras** para implementar el **modelo System Setup**.

Considere usar pseudoestados **Choice** de **Itemis Create** (https://www.itemis.com/en/products/itemis-create/documentation/user-guide/quick_ref_choices#quick_ref_choices).

Paso 04: **Generar** la **imagen** del **modelo** (**File** > **Save As Image File …**).

En su PC **crear** una carpeta **system_setup_menu**, **copiar** allí los archivos generados (**.ysc** y **.png**).

Paso 05: **Almacenar** la carpeta generada en la actividad **system_setup_menu**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

TP3 – Actividad 04 – System Setup Menu Statechart (C coding)

Objetivos

Usar STM32CubeIDE

Generar proyecto **STM32**

Editar/compilar/depurar/portar programas en **C**

Implementar Patrones de Diseño de Software

Diagramas de Estado – Codificación en C (System Setup Menu Statechart)

Garantizar cumplimiento de restricciones temporales del **ejecutor cíclico**

Usar Itemis Create **Desktop**

Generar proyecto **Itemis Create**
Editar/simular/depurar **diagrama de estado** (**modelo**): **System Setup Meni Statechart** (Modeling)

Usar Gemini

Analizar/explicar código **fuente**

Paso 01: **Generar** un nuevo **proyecto STM32**: **descargar** el archivo **tdse-tp3_03-system_setup_menu.zip** e **importar** el **proyecto STM32** que contiene: **tdse-tp3_03-system_setup_menu**.

Compilar el **proyecto STM32** (**STM32CubeIDE**) importado: **tdse-tp3_03-system_setup_menu**.

Desconectar placa **NUCIEO-F103RB** de PC.

Conectar el **LCD Display y Teclado membrana 1 x 4 teclas** a la placa **NUCLEO-F103RB** como en la imagen al pie.

. No olvide conectar y ajustar el **Trimpot** (**LINEAL - 10K**) a contrast adjustment (pin: **VO**)
Tenga cuidado al conectar las líneas de **backlight** (pines: **A** & **K**

Conectar placa **NUCIEO-F103RB** a PC.

Confirmar mediante la depuración, al ejecutar el programa (**toolbar button** > [**Resume** (**F8**)]):

Que **task_system_init**(), envía a **LCD Display interface** los mensajes:

Línea 1: **"task_system_mode"**
Línea 2: **" NORMAL "**

Que al mantener oprimido el botón **B1 USER** (**Blue**) **PushButton**, el led **LD2 Green Led** se enciende. Mientras que al soltarlo, el led se apaga.

Que al oprimir el botón **B1 USER** (**Blue**) **PushButton**, **task_system_update**(), envía a **LCD Display interface** los mensajes:

Línea 1: **"task_system_mode"**
Línea 2: **" NORMAL "**

o

Línea 1: **"task_system_mode"**
Línea 2: **" SETUP "**

y que dichos mensajes se ven en el **LCD Display**.

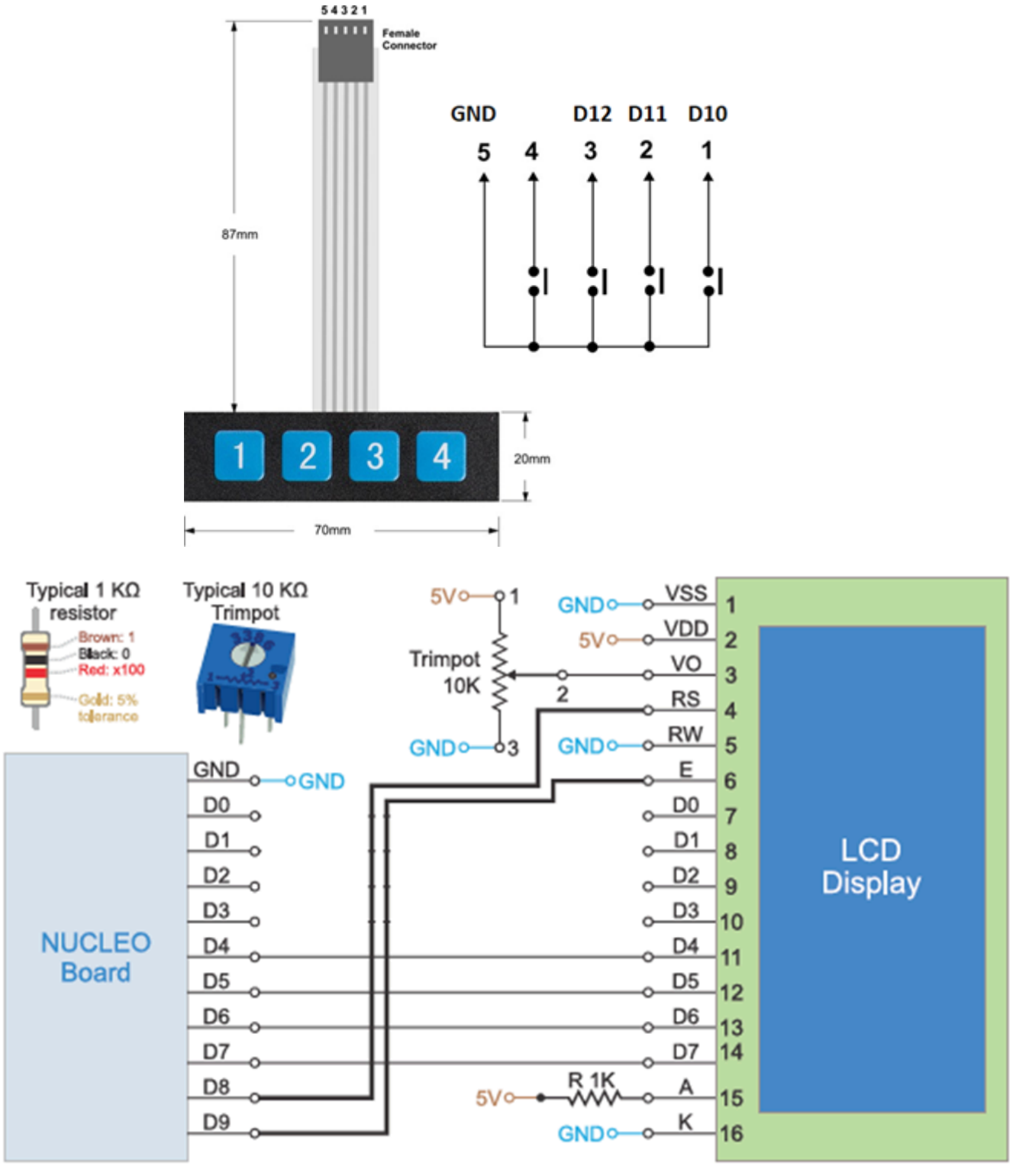


Figure 6.3 Diagram of the connections between the character LCD display and the NUCLEO board using GPIOs.

Paso 02: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **tdse-tp3_04-display.md**.

Paso 03: Modificar el código fuente, del nuevo **proyecto STM32**, **codificar** en **C** el Diagrama de Estado del **modelo system_setup_menu**, tal como se describe en: **system_setup_menu.jpg**. generado en el **TP3 – Actividad 03 – Paso 04**.

Para **Editar**, en **Project Explorer** view:

…\tdse_workspace\tdse-tp3_03-system_setup_menu\app\src\task_sensor.c (double-click)

…\tdse_workspace\tdse-tp3_03-system_setup_menu\app\inc\task_sensor_attribute.h (double-click)

…\tdse_workspace\tdse-tp3_03-system_setup_menu\app\src\task_system.c (double-click)

…\tdse_workspace\tdse-tp3_03-system_setup_menu\app\inc\task_system_attribute.h (double-click)

Paso 04: **Depurar** el nuevo **proyecto STM32** (**STM32CubeIDE**):

Confirmar mediante la depuración, el correcto funcionamiento.

Luego de varias ejecuciones de **app_update()**, **leer** y **almacenar** los valores de **task_dta_list[index]** (**indicar** unidad de medida), en**: …\tdse_workspace\tdse-tp3_02-porting_c_code\tdse-tp3_04-display.md**.

Analizar si cumplen con las restricciones temporales del **ejecutor cíclico**

Paso 05: **Almacenar** el **proyecto STM32**: **tdse-tp3_02-portng_c_code** y el/los archivo/s de la actividad: **tdse-tp3_04-display.md**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.
